

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №2
Дисциплина: «Методы поисковой оптимизации»

Работу выполнили: _____ Долгов М.И. и Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Нигодин Е.А.

Краснодар
2026

Тема. Симплекс-метод.

Цель. Реализовать приложение с графическим интерфейсом, демонстрирующее применение симплекс-метода для решения задачи квадратичного программирования.

Квадратичное программирование. Задачей квадратичного программирования (КП) называется задача оптимизации с квадратичной целевой функцией и линейными ограничениями. Задача КП, в которой целевая функция подлежит минимизации, имеет вид:

$$\begin{aligned} f(x) &= \sum_{j=1}^n c_j x_j + \sum_{j=1}^n \sum_{k=1}^n d_{jk} x_j x_k \rightarrow \min, \\ \sum_{j=1}^n a_{ij} x_j &\leq b_i, \quad i = \overline{1, m}, \\ x_j &\geq 0, \quad j = \overline{1, n}. \end{aligned}$$

Допустимое множество X является выпуклым, матрица $D = [d_{jk}]$ предполагается симметричной неотрицательно определённой. При этом $f(x)$ является выпуклой на X и задача КП является частным случаем задачи выпуклого программирования.

Для решения задачи КП используются необходимые условия Куна—Таккера, которые в силу выпуклости задачи КП оказываются также достаточными для установления наличия решения.

Предварительно составляется функция Лагранжа:

$$L(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i g_i(x),$$

$$\text{где } g_j(x) \equiv \sum_{j=1}^n a_{ij} x_j - b_i.$$

Условия Куна—Таккера имеют следующий вид:

$$\frac{\partial L(x, \lambda)}{\partial x_j} \geq 0, \quad j = \overline{1, n}; \quad (3.1)$$

$$x_j \geq 0, \quad j = \overline{1, n}; \quad (3.2)$$

$$x_j \frac{\partial L(x, \lambda)}{\partial x_j} = 0, \quad j = \overline{1, n}; \quad (3.3)$$

$$\frac{\partial L(x, \lambda)}{\partial \lambda_i} \leq 0, \quad i = \overline{1, m}; \quad (3.4)$$

$$\lambda_i \geq 0, \quad i = \overline{1, m}; \quad (3.5)$$

$$\lambda_i \frac{\partial L(x, \lambda)}{\partial \lambda_i} = 0, \quad i = \overline{1, m}. \quad (3.6)$$

Соотношения (3.3) и (3.6) определяют условия дополняющей нежёсткости.

Непосредственное решение данной системы очень громоздко. Поэтому используется следующий приём. Неравенства (3.1) и (3.4) преобразуют в равенства, вводя соответственно две группы дополнительных переменных, удовлетворяющих требованиям неотрицательности:

$$v_j, j = \overline{1, n}, \text{ и } w_i, i = \overline{1, m},$$

В результате от системы (3.1)-(3.6) переходят к следующей системе:

$$\frac{\partial L(x, \lambda)}{\partial x_j} - v_j = 0, \quad j = \overline{1, n}; \quad (3.7)$$

$$x_j \geq 0, \quad v_j \geq 0, \quad j = \overline{1, n}; \quad (3.8)$$

$$x_j v_j = 0, \quad j = \overline{1, n}; \quad (3.9)$$

$$\frac{\partial L(x, \lambda)}{\partial \lambda_i} + w_i = 0, \quad i = \overline{1, m}; \quad (3.10)$$

$$\lambda_i \geq 0, \quad w_i \geq 0, \quad i = \overline{1, m}; \quad (3.11)$$

$$\lambda_i w_i = 0, \quad i = \overline{1, m}. \quad (3.12)$$

Система уравнений (3.7) и (3.10) содержит $(m + n)$ линейных уравнений с $2(m + n)$ неизвестными. Таким образом, исходная задача эквивалентна задаче нахождения допустимого, т.е. удовлетворяющего требованиям неотрицательности (3.8) и (3.11), базисного решения системы линейных уравнений (3.7) и (3.10), удовлетворяющего также условиям дополняющей нежёсткости (3.9) и (3.12). Так как задача КП является выпуклой задачей оптимизации, то допустимое решение, которое удовлетворяет всем этим условиям, является оптимальным.

Для нахождения допустимого базисного решения системы линейных уравнений может быть применён метод искусственных переменных, используемый в линейном программировании (ЛП) для определения начального базиса. При этом в уравнения системы (3.7), (3.10), в которых знаки дополнительных переменных v_i или w_i совпадают со знаками свободных членов, вводятся неотрицательные искусственные переменные, знаки которых не совпадают со знаками соответствующих свободных членов:

$$z_l, l = \overline{1, l_{\max}}, \quad l_{\max} \leq m + n,$$

Затем решается вспомогательная задача ЛП при ограничениях (3.7)-(3.12) с введёнными неотрицательными искусственными переменными:

$$F(z) = \sum_l z_l \rightarrow \min$$

Для решения этой задачи используется симплекс-метод. В процессе решения необходимо учитывать условия дополняющей нежёсткости (3.9) и (3.12). Выполнение этих условий означает, что x_j и v_j , λ_i и w_i не могут быть положительными одновременно, т.е. переменную x_j нельзя сделать базисной, если v_j является базисной и принимает положительное значение, также и λ_i с w_i .

В результате решения вспомогательной задачи могут быть два случая:

1. $\min F(z) = 0$, т.е. все искусственные переменные выведены из базиса. Полученное оптимальное допустимое базисное решение вспомогательной задачи ЛП является допустимым базисным решением системы (3.7)-(3.12) и, следовательно, решением задачи КП.

2. $\min F(z) > 0$, т.е. среди базисных остались искусственные переменные. Это означает, что система (3.7)-(3.12) не имеет допустимого базисного решения и, следовательно, задача КП не имеет решения.

Алгоритм.

Шаг 1. Ограничения задачи КП преобразуются к виду:

$$g_i(x) \leq 0, \quad i = 1, m.$$

Шаг 2. Составляется функция Лагранжа:

$$L(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i g_i(x).$$

Шаг 3. Находятся частные производные и составляются условия Куна—Таккера (3.1)-(3.6).

Шаг 4. Посредством введения дополнительных переменных неравенства преобразуются в равенства. В результате получается система (3.7)-(3.12).

Шаг 5. Вводятся искусственные переменные z_i , составляется вспомогательная задача ЛП минимизации $F(z)$ при ограничениях (3.7)-(3.12) с введёнными неотрицательными искусственными переменными.

Шаг 6. Решается вспомогательная задача ЛП симплекс-методом с учётом условий дополняющей нежёсткости.

Если в результате решения $\min F(z) = 0$, то оптимальное допустимое базисное решение вспомогательной задачи определяет решение задачи x^* задачи КП.

Если $\min F(z) > 0$, то задача КП не имеет решения.

1. Программа представляет собой desktop-приложение, реализованное на языке Python с использованием фреймворка PySide6 и библиотеки rpyqtgraph для визуализации. Приложение предназначено для демонстрации алгоритмов оптимизации функций в 3D-пространстве (рисунок 1).

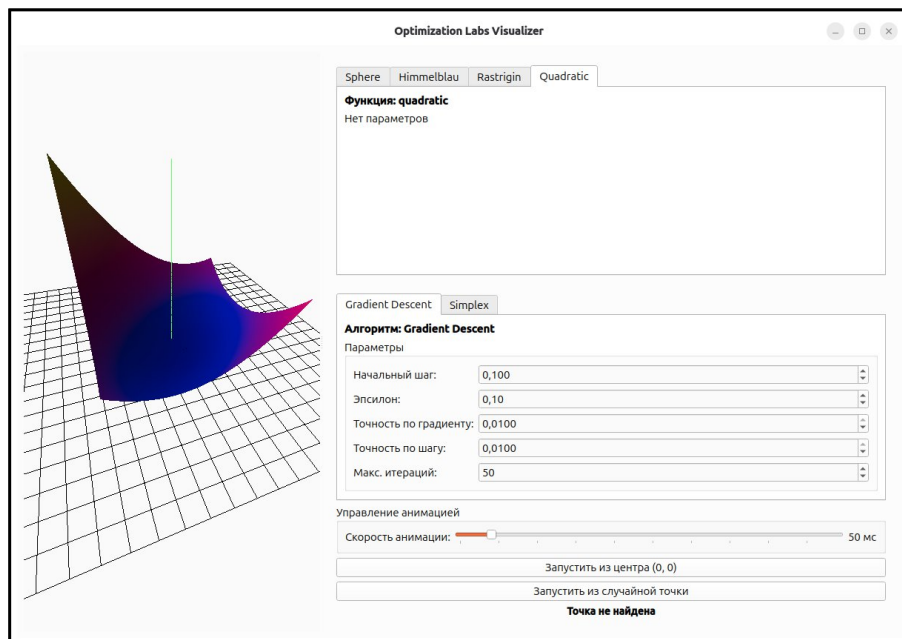


Рисунок 1 — Главное окно приложения

2. Доступные функции: Sphere, Himmelblau, Rastrigin и Quadratic (рисунок 2).

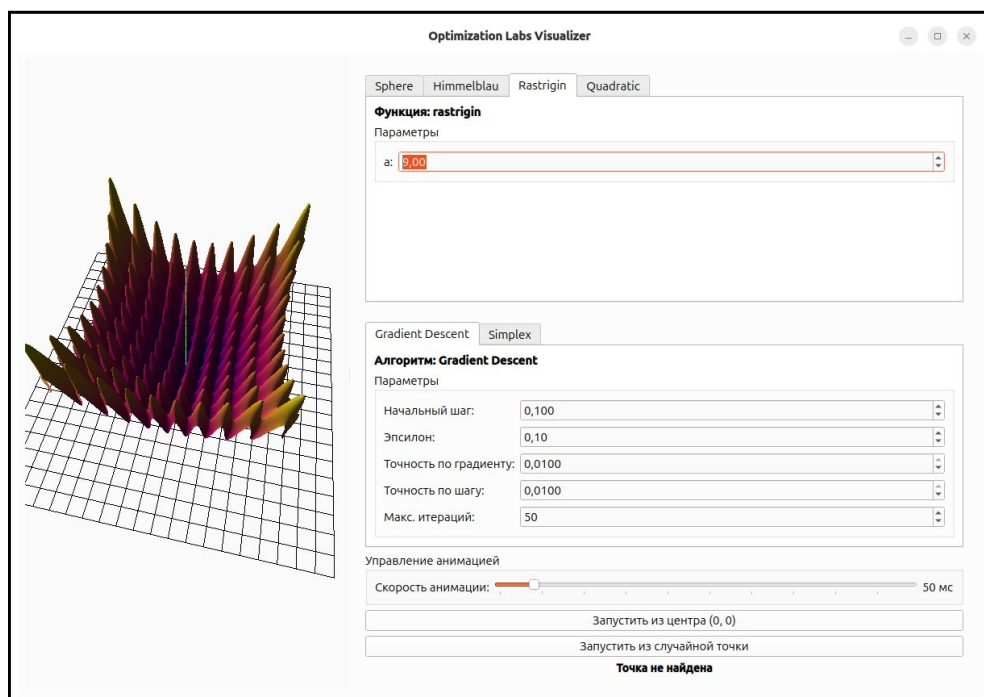


Рисунок 2 — Доступные функции

3. Доступные алгоритмы оптимизации: градиентный спуск с постоянным шагом и симплекс-метод (рисунок 3).

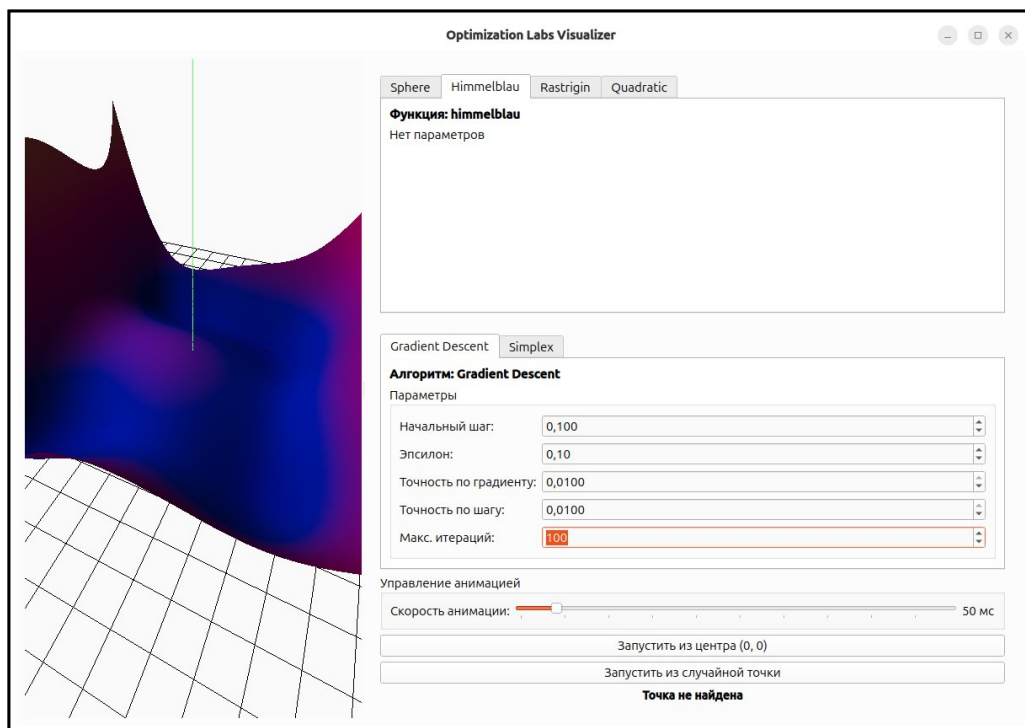


Рисунок 3 — Доступные алгоритмы оптимизации

4. Запустим алгоритм поиска минимума с ограничениями функции Quadratic из случайной точки (рисунок 4).

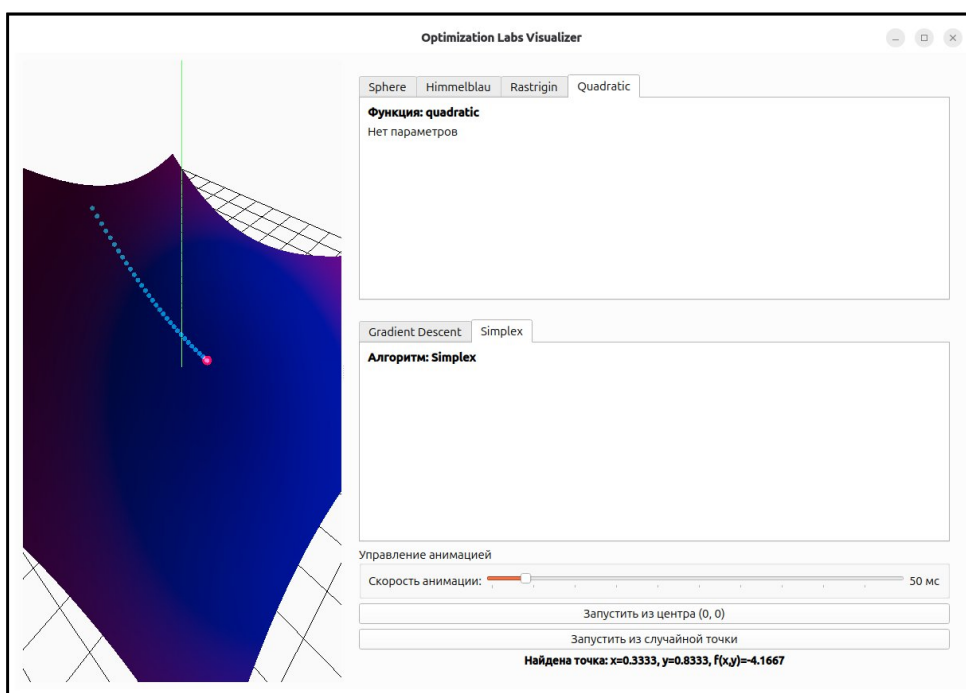


Рисунок 4 — Запуск алгоритма оптимизации

5. Запустим алгоритм поиска минимума функции из точки $(0, 0)$ (рисунок 5).

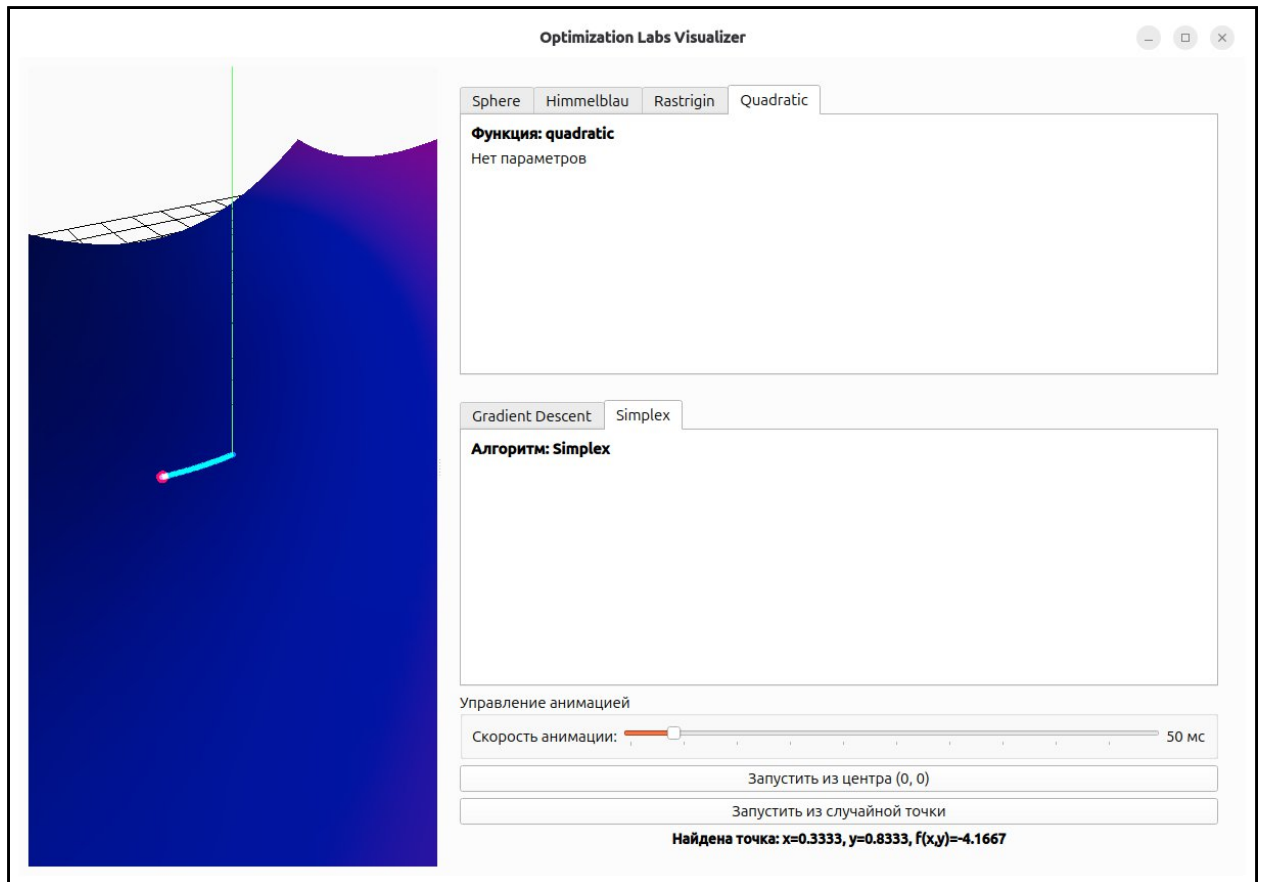


Рисунок 5 — Запуск алгоритма

Вывод. В ходе данной лабораторной работы было реализовано приложение с графическим интерфейсом, демонстрирующее применение симплекс-метода для решения задачи квадратичного программирования.

Листинг программы.

```
import numpy as np

class SimplexSolver:
    """
    Реализация симплекс-метода для решения вспомогательной задачи ЛПП
    с учётом всех условий
    """

    def __init__(self):
        self.tables = []
        self.basis_names = []
        self.solution = None
        self.f_opt = None
        self.iteration = 0
        self.var_names = ['x1', 'x2', 'λ', 'v1', 'v2', 'w', 'z1', 'z2']
        self.complementary_pairs = [('x1', 'v1'), ('x2', 'v2'), ('λ', 'w')]
```

```

def solve_lab2_problem(self):
    """
    Решает задачу из лабораторной работы №2
    """
    A = np.array([
        [4, 4, 2, 1, -1, 0, 0, 1, 0],
        [6, 2, 4, 2, 0, -1, 0, 0, 1],
        [2, 1, 2, 0, 0, 0, 1, 0, 0]
    ], dtype=float)

    F = np.array([10, 6, 6, 3, -1, -1, 0, 0, 0], dtype=float)
    self.basis_names = ['z1', 'z2', 'w']
    self.iteration = 0

    self._save_table(A, F, self.basis_names)

    while True:
        self.iteration += 1

        # ШАГ 1: Проверка оптимальности
        if np.all(F[1:] <= 1e-10):
            break

        # ШАГ 2: Выбор вводимой переменной с учётом условий
        entering_idx = None
        max_val = -np.inf
        for j in range(1, len(F)):
            if F[j] > 1e-10 and F[j] > max_val:
                var_name = self.var_names[j-1]
                if self._can_enter_basis(var_name, self.basis_names, A):
                    max_val = F[j]
                    entering_idx = j

        if entering_idx is None:
            break

        # ШАГ 3: Выбор выводимой переменной
        ratios = []
        for i in range(len(A)):
            if A[i, entering_idx] > 1e-10:
                ratio = A[i, 0] / A[i, entering_idx]
                ratios.append((ratio, i))

        if not ratios:
            raise Exception("Задача неограничена! Все отношения <= 0")

        min_ratio, leaving_row = min(ratios, key=lambda x: x[0])

        # ШАГ 4: Симплекс-преобразование
        A, F, self.basis_names = self._simplex_iteration(
            A, F, self.basis_names, leaving_row, entering_idx
        )

        self._save_table(A, F, self.basis_names)

    self._extract_solution(A, F)
    return self.solution, self.f_opt, self.tables

def _can_enter_basis(self, var_name, current_basis, A):
    """
    Проверяет, можно ли ввести переменную в базис
    с учётом условий
    """
    partner = None
    for p1, p2 in self.complementary_pairs:
        if var_name == p1:
            partner = p2
            break
        if var_name == p2:
            partner = p1
            break

    if partner is None:
        return True

    if partner in current_basis:
        partner_idx = current_basis.index(partner)
        partner_value = A[partner_idx, 0]
        if partner_value > 1e-8:

```



```

        return False

    return True

def _simplex_iteration(self, A, F, basis, pivot_row, pivot_col):
    """Выполняет одну итерацию симплекс-метода"""
    m, n = A.shape

    pivot = A[pivot_row, pivot_col]
    A[pivot_row] = A[pivot_row] / pivot

    for i in range(m):
        if i != pivot_row:
            factor = A[i, pivot_col]
            A[i] = A[i] - factor * A[pivot_row]

    factor = F[pivot_col]
    F = F - factor * A[pivot_row]

    entering_var = self.var_names[pivot_col - 1]
    new_basis = basis.copy()
    new_basis[pivot_row] = entering_var

    return A, F, new_basis

def _save_table(self, A, F, basis):
    """Сохраняет симплекс-таблицу"""
    self.tables.append({
        'iteration': self.iteration,
        'A': A.copy(),
        'F': F.copy(),
        'basis': basis.copy()
    })

def _extract_solution(self, A, F):
    """Извлекает решение из финальной симплекс-таблицы"""
    var_indices = {'x1': 0, 'x2': 1, 'λ': 2, 'v1': 3, 'v2': 4, 'w': 5, 'z1': 6, 'z2': 7}
    x = np.zeros(8)

    for i, var_name in enumerate(self.basis_names):
        if var_name in var_indices:
            idx = var_indices[var_name]
            x[idx] = A[i, 0]

    f_val = 2*x[0]**2 + 2*x[0]*x[1] + 2*x[1]**2 - 4*x[0] - 6*x[1]

    self.solution = (x[0], x[1])
    self.f_opt = f_val

def simplex_algorithm(func, x0, y0, **func_params):
    """
    Функция для визуализации метода с симплексом
    """
    solver = SimplexSolver()
    solution, f_opt, tables = solver.solve_lab2_problem()

    yield (x0, y0)

    steps = 30
    for i in range(1, steps + 1):
        alpha = i / steps
        curr_x = x0 + alpha * (solution[0] - x0)
        curr_y = y0 + alpha * (solution[1] - y0)
        yield (curr_x, curr_y)

    yield (solution[0], solution[1])

```